

Word Embeddings, Attention and Transformers

Word Representation

- NLP has been revolutionized by deep learning and especially by RNNs and deep RNNs.
- Word embeddings is a way of representing words. It lets your algorithm automatically understand the analogies between words like "king" and "queen".
- So far we have defined our language by a vocabulary and one-hot encodings.

Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$

Word Representation

- "I want a glass of orange juice"
- "I want a glass of apple _____"
- The model won't easily predict juice here if it wasn't trained on it
- It will be nice if we can learn a featurized representation with each of these words: man, woman, king, queen, apple, and orange?
- Each word will have a, for example, 300 features with a type of float point number

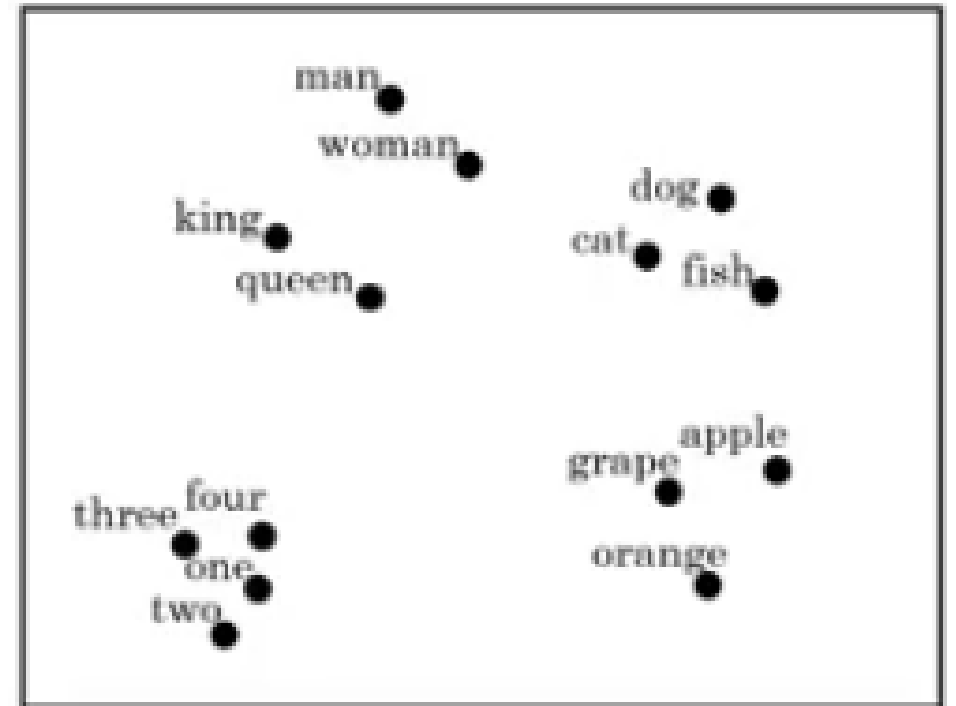
Word Embeddings

Now, if we return to the examples we described again:

"I want a glass of orange _____"

I want a glass of apple _____

- Orange and apple now share a lot of similar features which makes it easier for an algorithm to generalize between them.
- We call this representation Word embeddings.

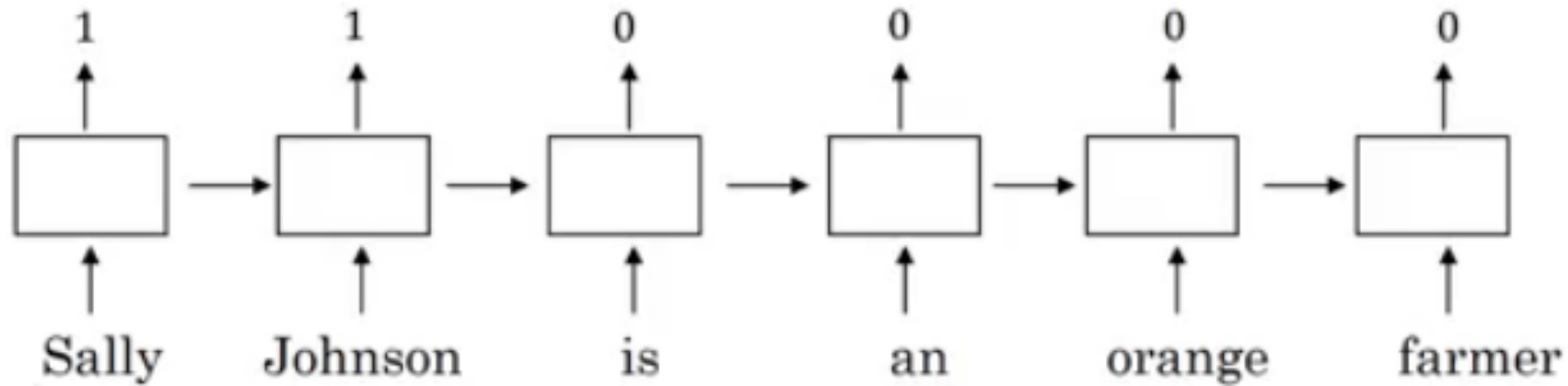


Featurized Representation of Word Embeddings

	Man	Woman	King	Queen	Apple	Orange
Gender	-1	1	-0.95	0.95	0	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97

Embeddings are learned/stored as matrix E

Example: Named Entity Recognition



Robert Lin is a apple farmer

Transfer Learning and Word Embeddings

- Learn word embeddings from large text corpus (1-100 billion of words).
 - Or download pre-trained embedding online.
- Transfer embedding to new task with the smaller training set (say, 100k words).
- Optional: continue to finetune the word embeddings with new data.
 - You bother doing this if your smaller training set (from step 2) is big enough.

Properties of Word Embeddings

Analogies

- One of the most fascinating properties of word embeddings is that they can also help with analogy reasoning
- Can we conclude this relation:
 - Man ==> Woman
 - King ==> ??

Lets subtract eMan from eWoman.

This will equal the vector [-2 0 0 0]

Similar eKing - eQueen = [-2 0 0 0]

	Man	Woman	King	Queen	Apple	Orange
Gender	-1	1	-0.95	0.95	0	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97

Similarity Measure

$$\text{sim}(e_w, e_{king} - e_{man} + e_{woman})$$

Cosine Similarity

Euclidean distance

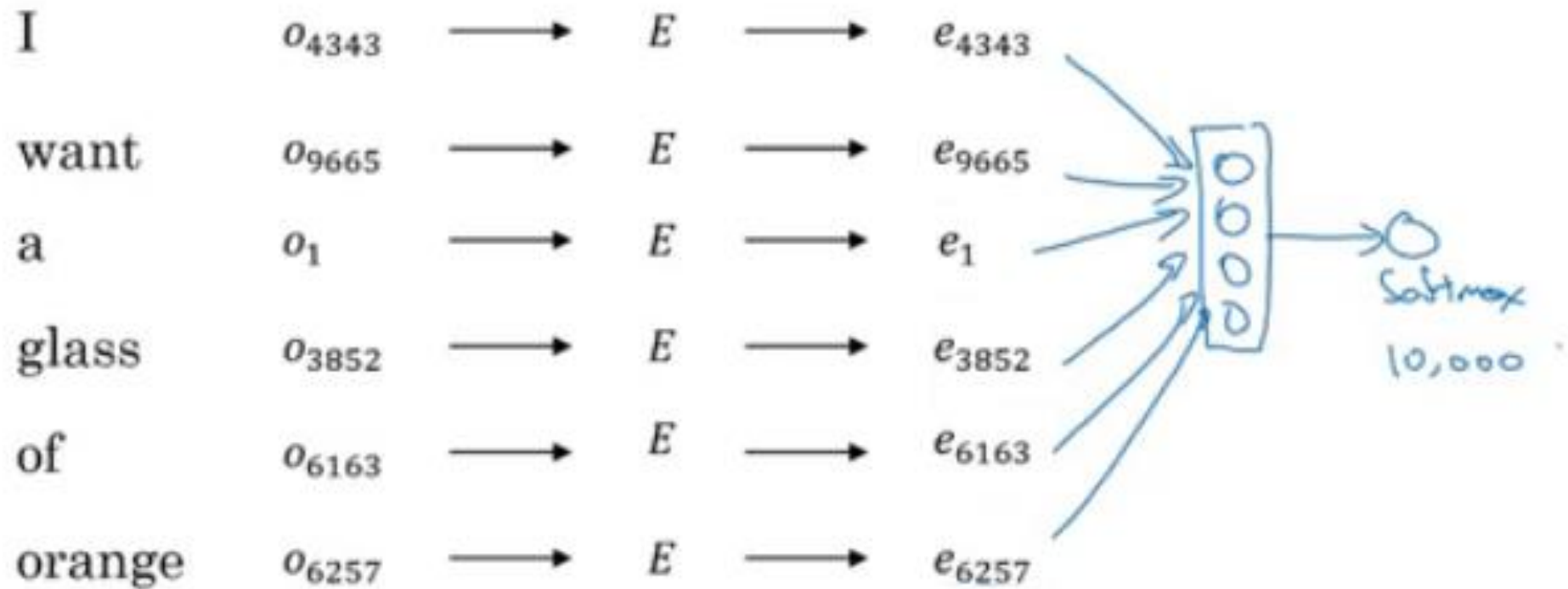
Man:Woman as Boy:Girl

Ottawa:Canada as Nairobi:Kenya

Big:Bigger as Tall:Taller

Yen:Japan as Ruble:Russia

Learning Word Embeddings



Other Context Pairs

I want a glass of orange **juice**

- To learn **juice**, choices of **context** are:
- Last 4 words.
 - We use a window of last 4 words (4 is a hyperparameter), "a glass of orange" and try to predict the next word from it.
- 4 words on the left and on the right.
 - "a glass of orange" and "to go along with"
- Last 1 word.
 - "orange"
- Nearby 1 word.
 - "glass" word is near juice.

Skip gram model

"I want a glass of orange juice to go along with my cereal"

- We will choose context and target.
- The target is chosen randomly based on a window with a specific size

Context	Target	How far
orange	juice	+1
orange	glass	-2
orange	my	+6

Skip Gram Model

- Vocabulary size = 10,000 words
- Let's say that the context word are c and target word is t
- We want to learn c to t
- We get e_c by E
- We then use softmax layer to get $P(t|c)$ which is $\hat{y}$
- Also we will use the cross-entropy loss function.

$$p(t|c) = \frac{e^{\theta_t^T e_c}}{\sum_{j=1}^{10,000} e^{\theta_j^T e_c}}$$

Sequence Models and Attention Mechanism

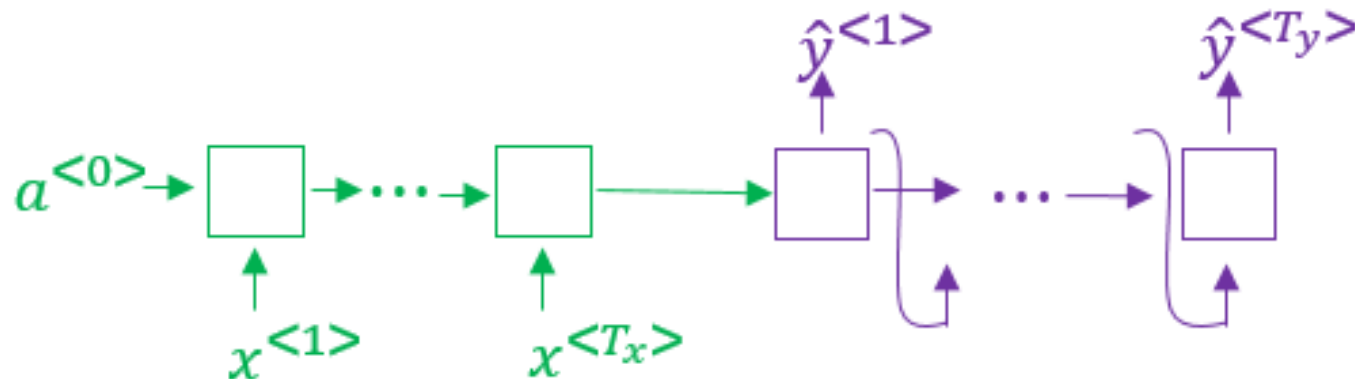
Sequence to sequence architecture

- Given this machine translation problem in which X is a French sequence and Y is an English sequence.

$x^{<1>} \quad x^{<2>} \quad x^{<3>} \quad x^{<4>} \quad x^{<5>}$
Jane visite l'Afrique en septembre

→ Jane is visiting Africa in September.

$y^{<1>} \quad y^{<2>} \quad y^{<3>} \quad y^{<4>} \quad y^{<5>} \quad y^{<6>}$

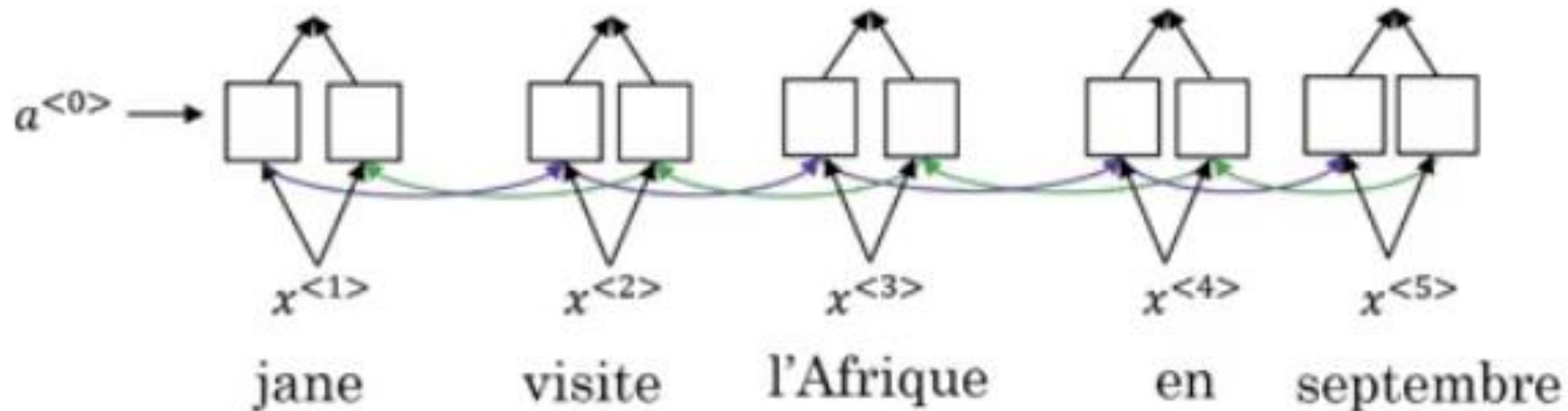


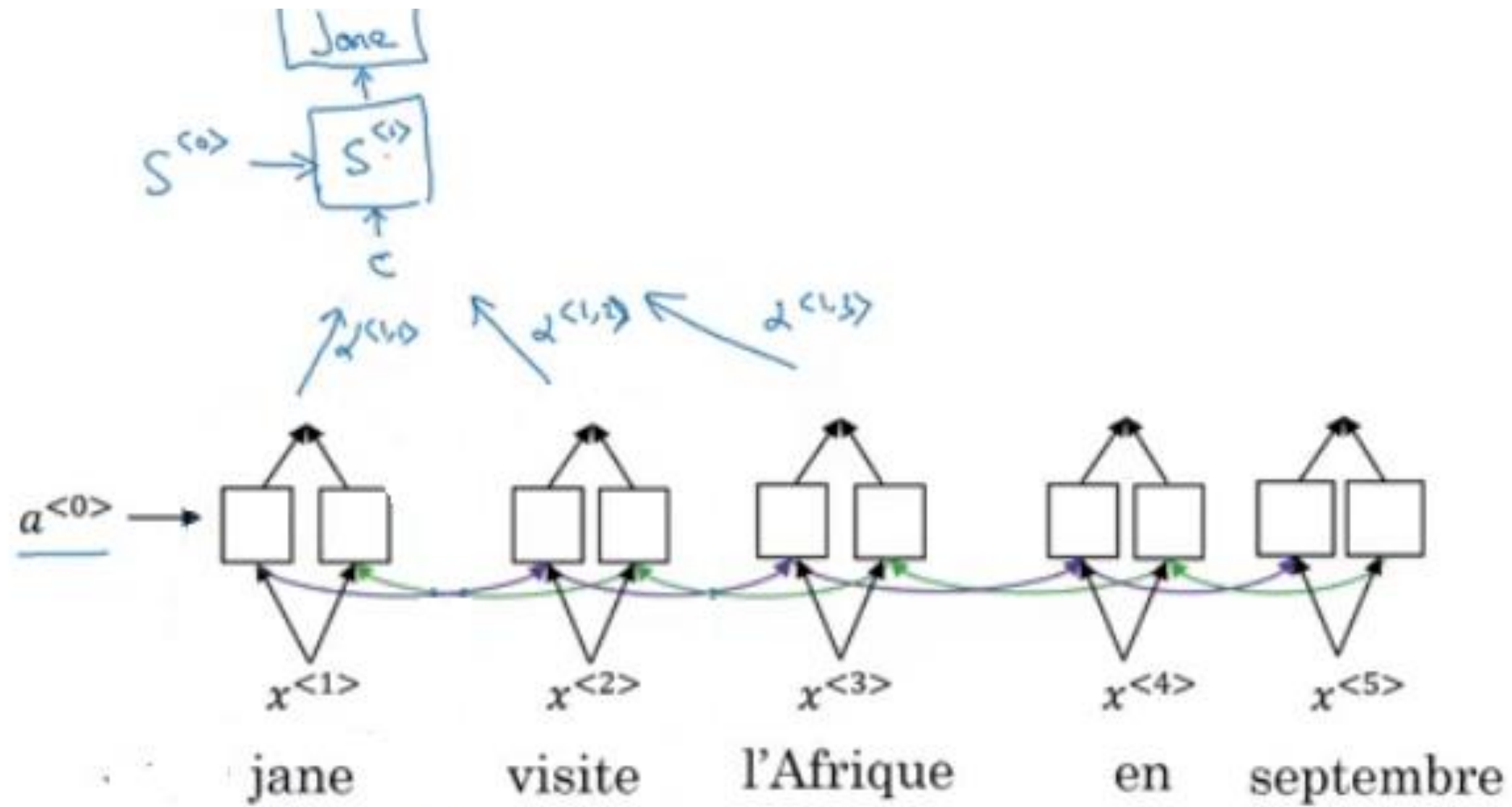
Attention Model Intuition

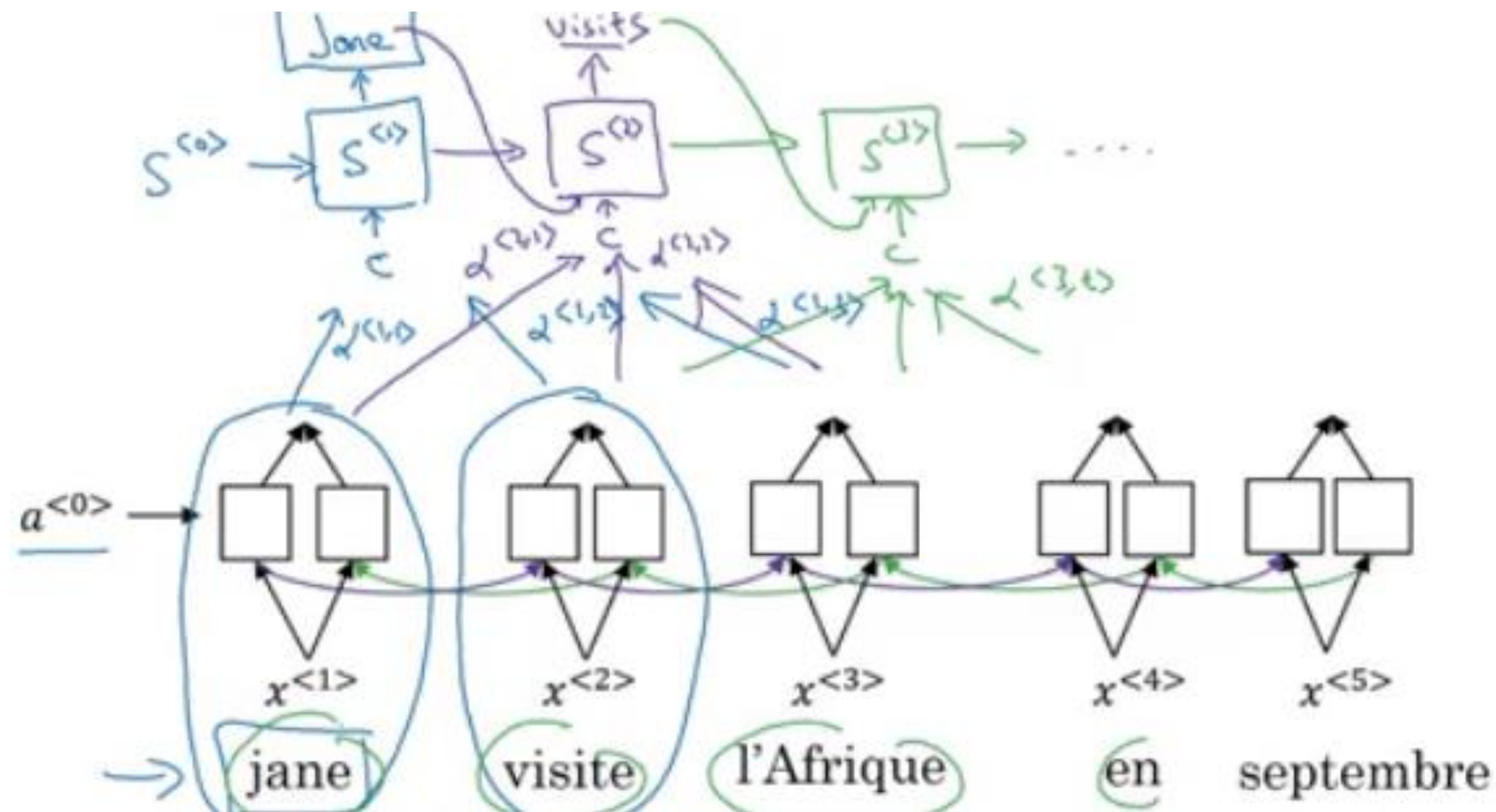
- The encoder should memorize this long sequence into one vector, and the decoder has to process this vector to generate the translation.
- A human translates a part at a time.
- The performance of this model decreases if a sentence is long.
- Attention model works like a human that looks at parts at a time.
- Significantly increase the accuracy even with longer sequence:

Attention Mechanism

- Encoder Architecture – bidirectional RNN
- Input French sentence to the encoder to generate a vector that represents the inputs.
- Make another RNN which is the decoder.
- Attention weights are used to specify which words are needed to generate a word. So to generate "jane" we will look at "jane", "visite", "l'Afrique"



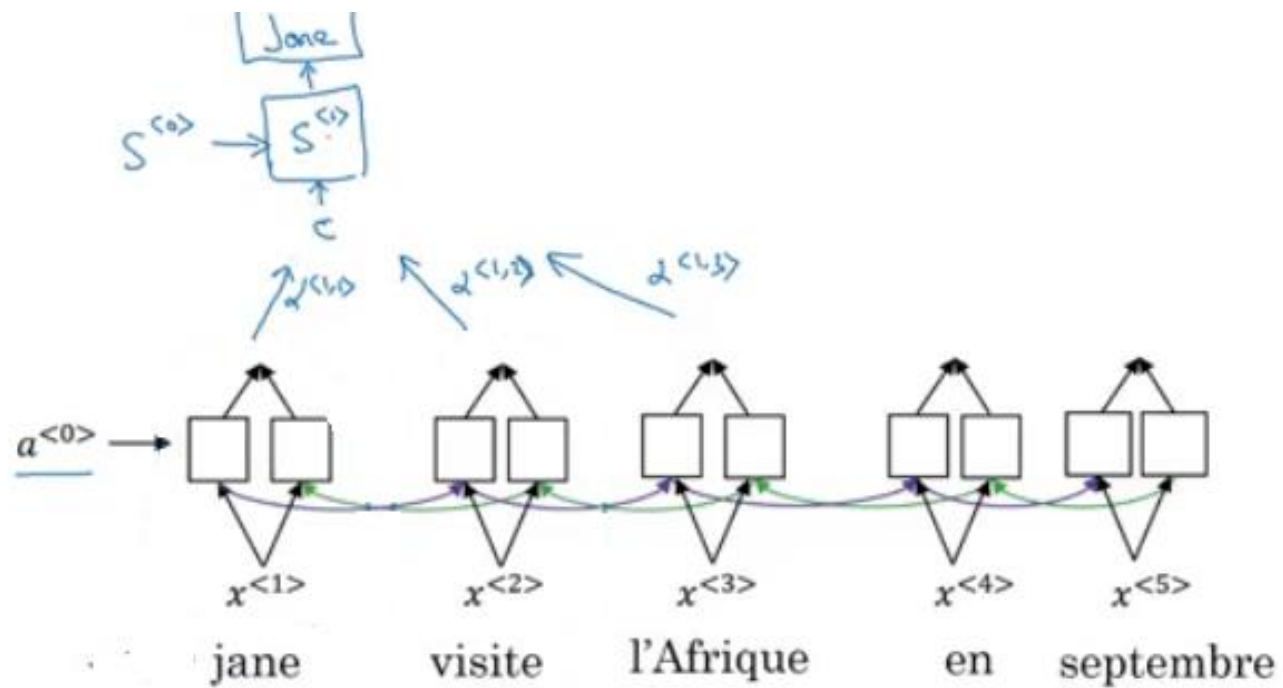




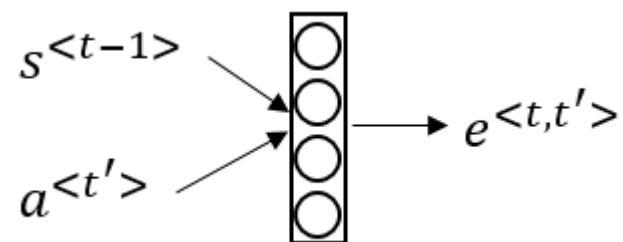
Attention Model

- Bidirectional RNN (most common is LSTMs) as encoder
- lets assume that $a^{<t'>}$ will include the both directions activations at time step t' .
- Decoder is an unidirectional RNN to produce the output using a context c
- c is computed using the attention weights(how much information does the output needs to look in $a^{<t'>}$)
- Sum of the attention weights for each element in the sequence should be 1 and can be computed using:

$$c^{<t>} = \sum_{t'} \alpha^{<t, t'>} a^{<t'>}$$

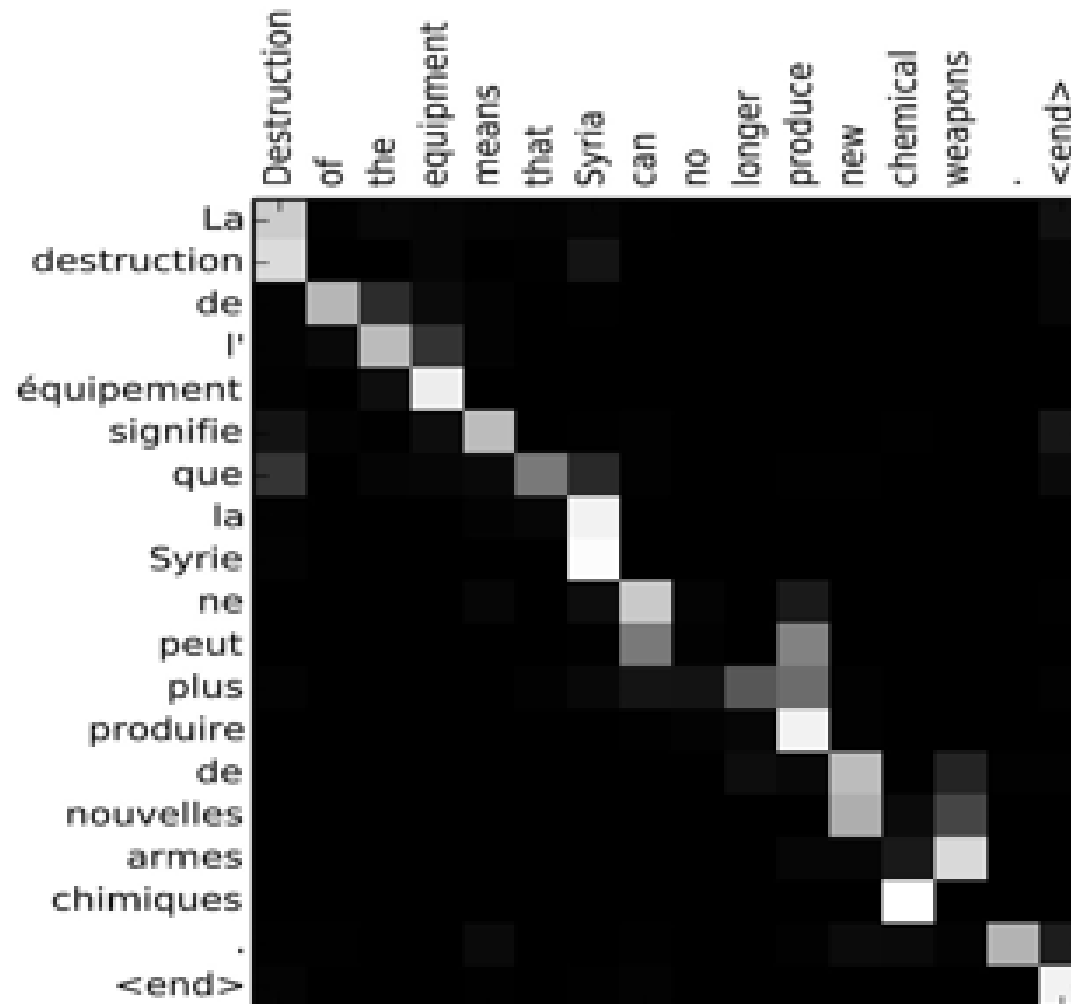


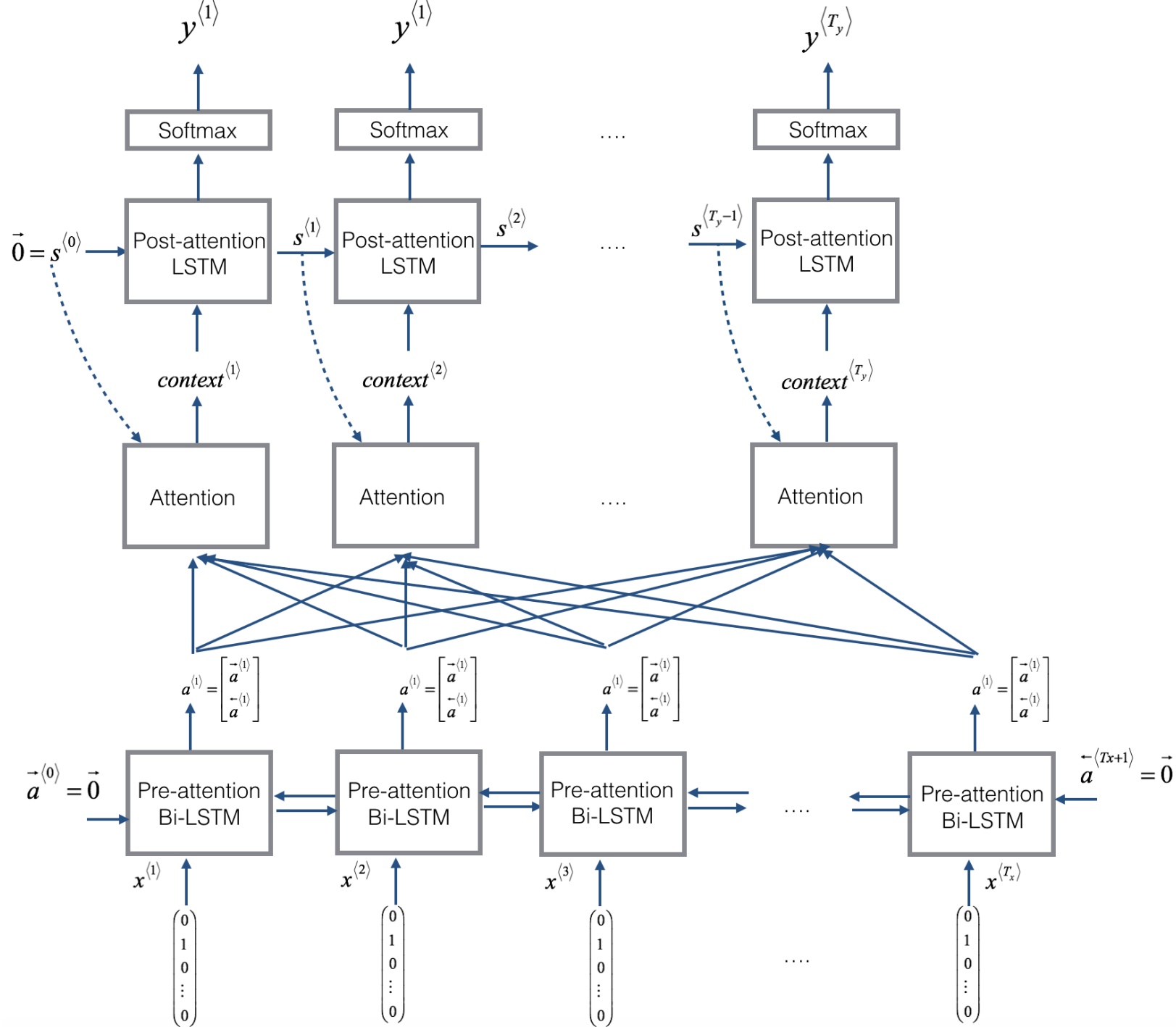
$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^{Tx} \exp(e^{<t,t'>})}$$



$$c^{<t>} = \sum_{t'} \alpha^{<t,t'>} a^{<t'>}$$

Visualization of attention weights





Transformers

Self Attention

$A(q, K, V)$ = attention-based vector representation of a word

RNN Attention

$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^{T_x} \exp(e^{<t,t'>})}$$

$x^{<1>}$
Jane

$x^{<2>}$
visite

$x^{<3>}$
l'Afrique

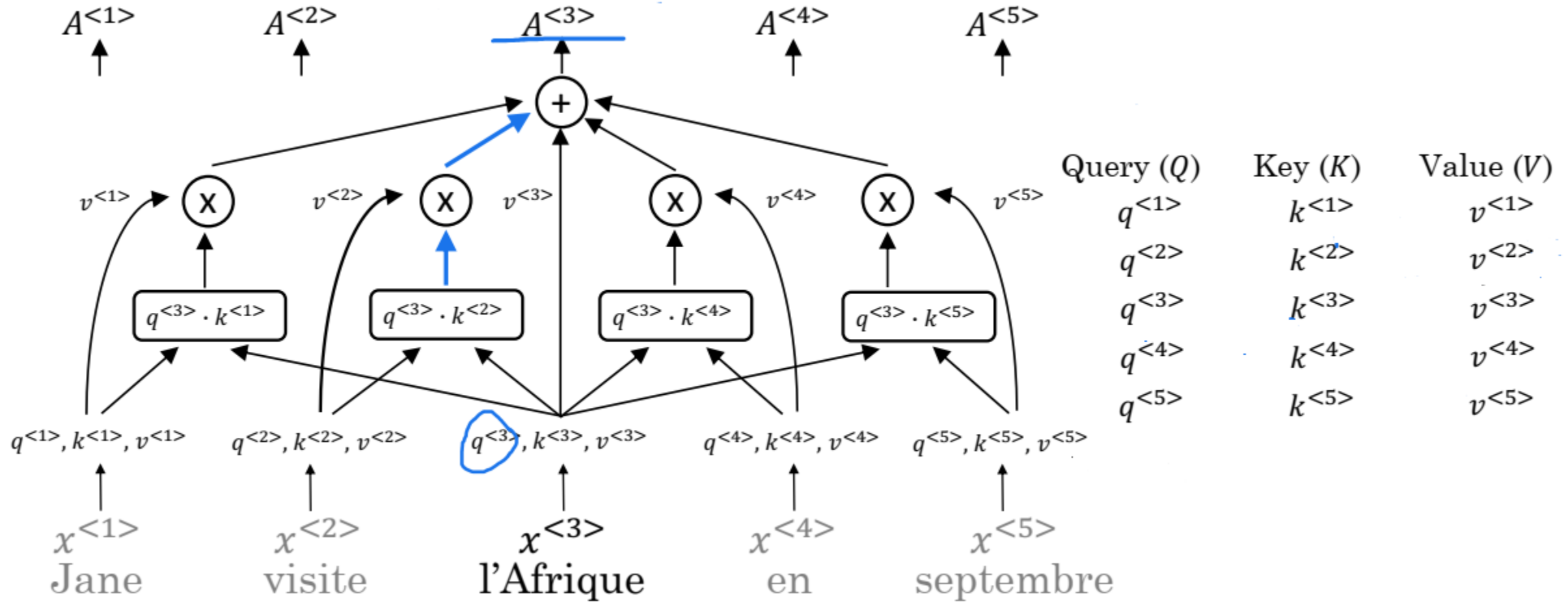
$x^{<4>}$
en

$x^{<5>}$
septembre

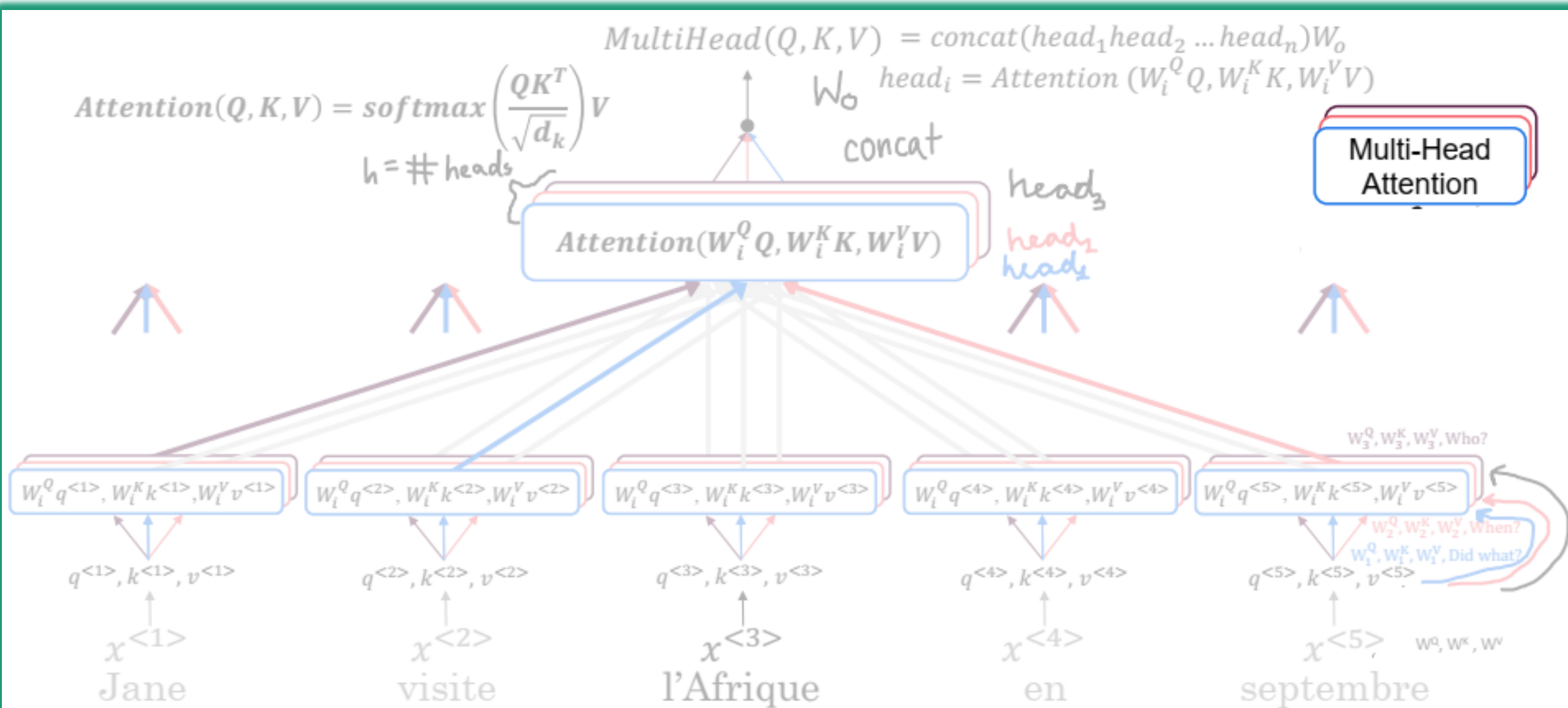
Transformers Attention

$$A(q, K, V) = \sum_i \frac{\exp(q \cdot k^{<i>})}{\sum_j \exp(q \cdot k^{<j>})} v^{<i>}$$

Self Attention



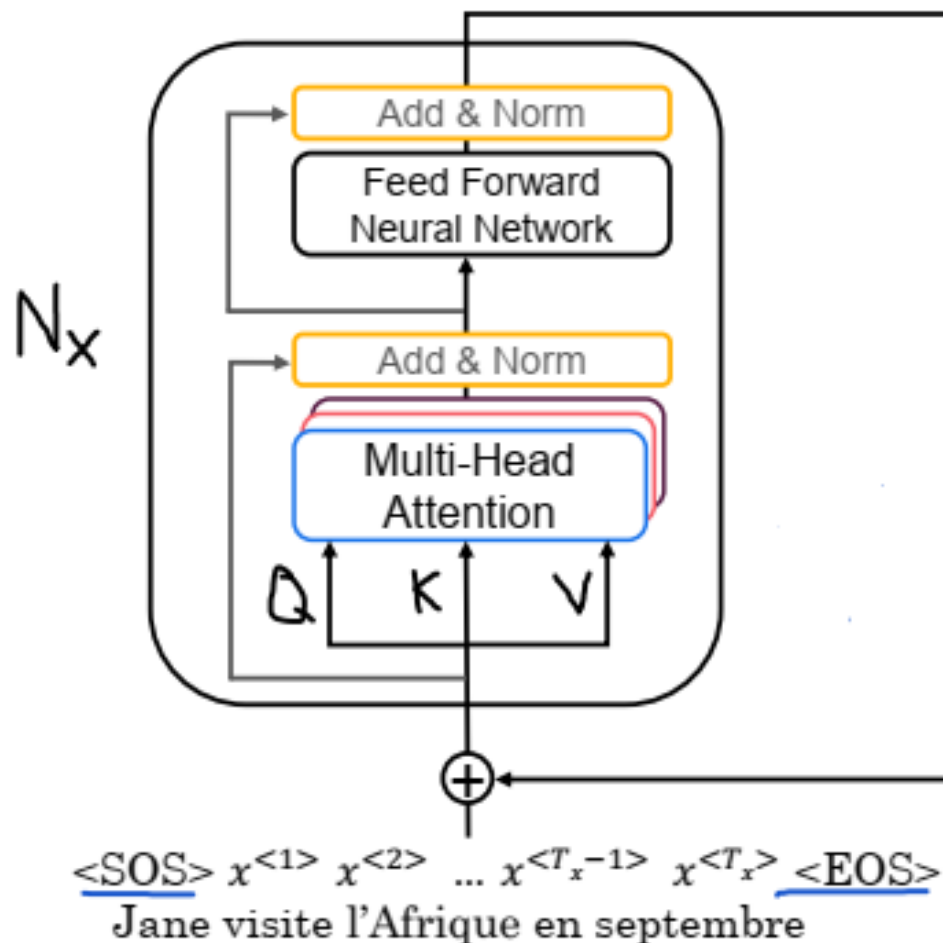
Multihead Attention



Transformer Details

<SOS> Jane visits Africa in September <EOS>

Encoder



Decoder

Positional Encoding

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

$\text{<SOS>} y^{<1>} y^{<2>} \dots y^{<T_y-1>} y^{<T_y>}$

